

Final Report

Student name: Boran Zhang (Bob)

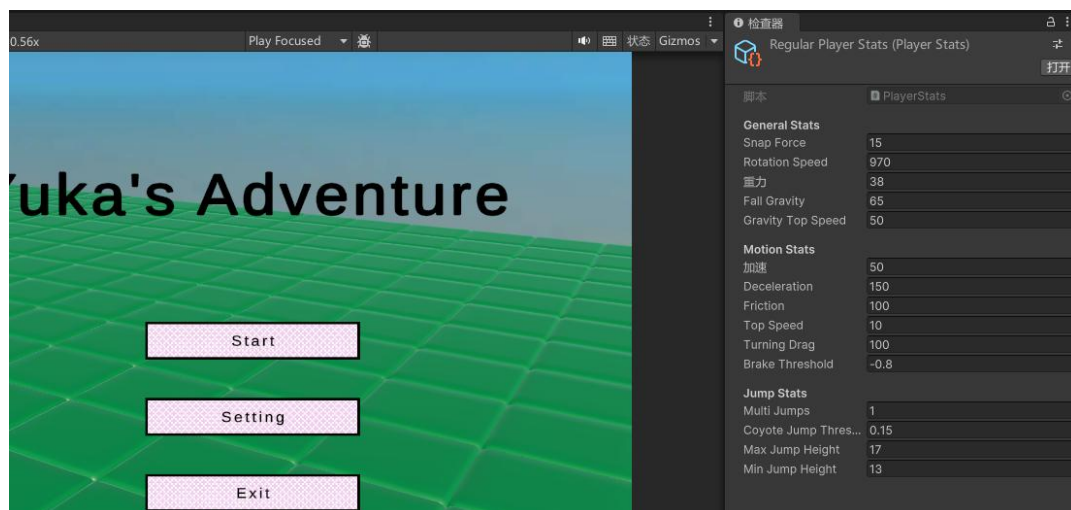
1. Design Choices

The core positioning of this project is the practice and demonstration of the 3C (Character, Control, Camera) system. All design decisions revolve around two principles: “feel-first” and “designer-friendly.”

At the gameplay level, I chose platformer-collector as the core gameplay framework. This genre demands the most comprehensive 3C systems—responsive characters, precise controls, and stable cameras—making it an ideal vehicle for demonstrating technical capabilities. The game objective is set as “collect water bottles → reach the goal → earn a star rating,” which is simple and intuitive, allowing players to quickly focus on the control experience itself.

For interaction design, I provided full keyboard/mouse input support. The camera uses a third-person follow perspective with free orbit control, while the velocity-driven rotation mechanism automatically adjusts the camera toward the movement direction during motion.

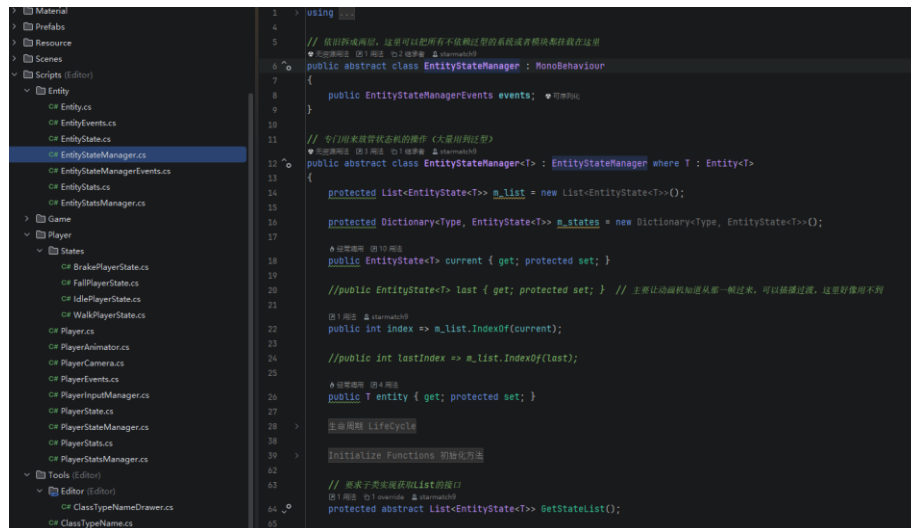
For data-driven design, I placed all feel parameters into the PlayerStats ScriptableObject, allowing designers to adjust values in the Inspector without touching code.



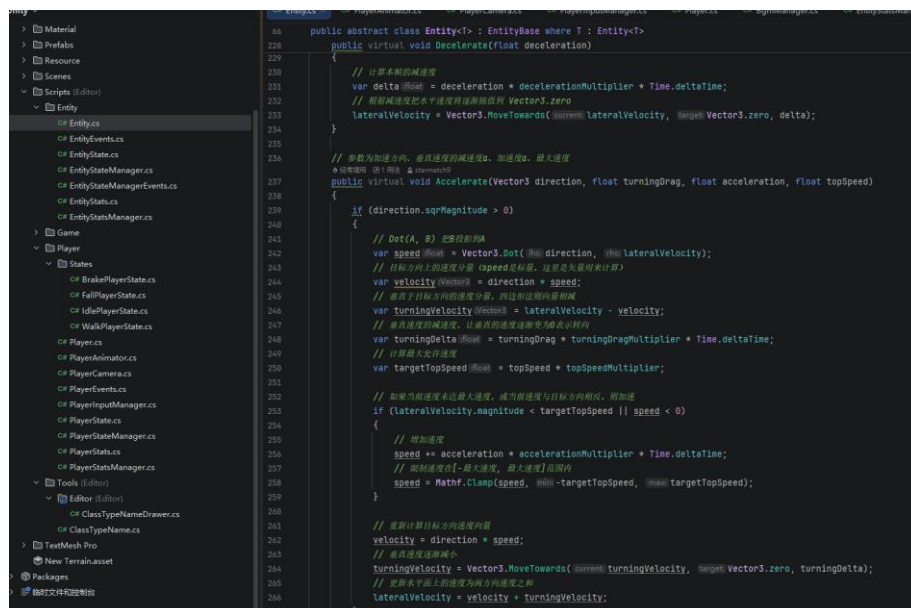
For editor experience, I developed the [ClassName] custom attribute and ClassNameDrawer, allowing designers to select state classes from a dropdown menu—completely eliminating typos from manual entry.

2. Technical Decisions

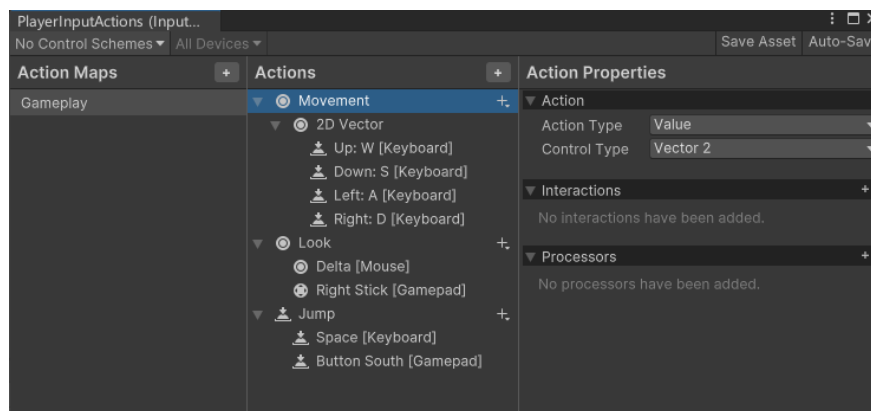
State Machine Architecture: Built on CRTP (Curiously Recurring Template Pattern). `Entity<T>` allows each subclass to hold its own type at compile time, eliminating runtime casting. `EntityState<T>` and `EntityStateManager<T>` support state switching by type, index, or instance, with events triggered during transitions. New states simply inherit from `PlayerState` and implement three methods—zero modifications to existing code.



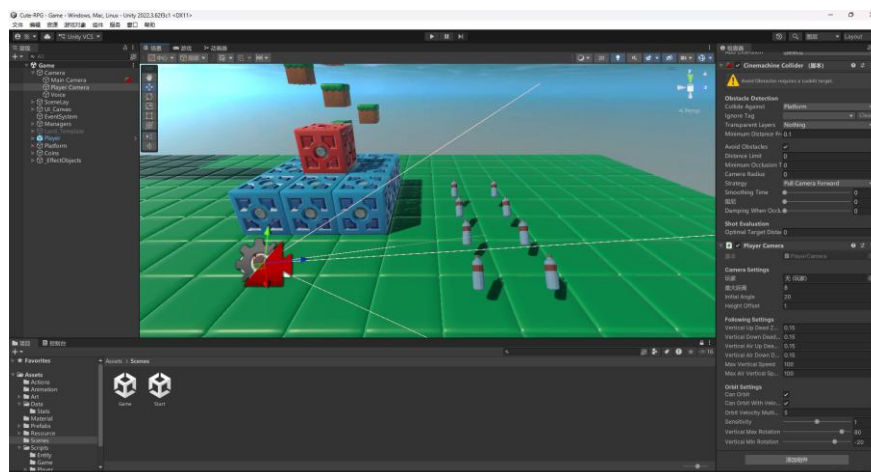
Physics Feel System: Independently implemented acceleration, turning drag, gravity, and friction calculations rather than relying on Unity's physics engine. This provides precise per-frame velocity control and allows all feel parameters to be independently tuned. The jump system implements Coyote Time, Jump Buffer, and variable jump height.



Input System: Uses Unity Input System with keyboard and mouse support. Cross deadzone processing with remapping smoothing delivers natural feel. Mouse uses Time.timeScale for instant response.

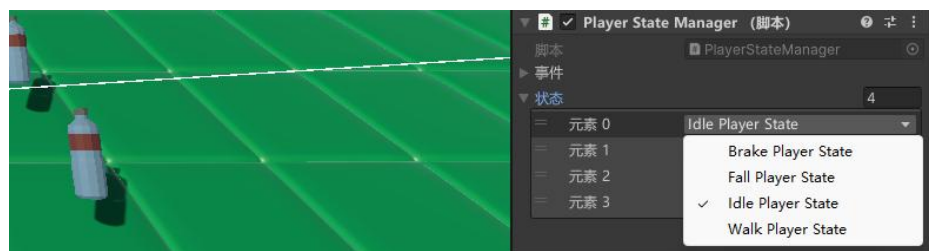


Camera System: Built on Cinemachine with three control layers: manual orbiting, vertical dead zone following (ground/air separation), and velocity-driven rotation.



Audio System: EffectManager singleton manages SFX playback. BGM uses coroutine-based loop with fade in/out. GlobalData holds references to avoid singleton lookup overhead.

Editor Tool: ClassNameDrawer scans for subclasses via reflection and generates a dropdown menu in the Inspector.



3. Problems and Limitations

Limited Content Completeness: Currently only one test level exists; lacks multi-level progression and difficulty curves. Overall playable content is limited.

Insufficient Gameplay Depth: Currently only features basic platforming and collection. Lacks enemies, time trials, hidden collectibles, or other replay-value elements.

Insufficient Feel Tuning: Parameters are set based on developer's subjective feel and haven't undergone sufficient user testing.

Low UI Polish: Only basic functionality; lacks unified art style and interactive feedback. Also lacks a main menu and onboarding flow.

4. Testing and Iteration

Throughout development, I conducted continuous manual testing:

Feel Testing: Adjusted acceleration, turning drag, jump height, and gravity through repeated playtesting. Early versions had abrupt startup feel; after lowering acceleration and introducing turning drag, movement became smooth. Coyote Time was set to 0.15 seconds.

Camera Testing: Early versions had excessive camera shake during jumps. After introducing ground/air separated dead zones—with air dead zone at 4.0—camera stability improved significantly.

State Machine Testing: Validated state transitions and event triggering through repeated switching. BrakePlayerState was introduced to solve the “teleporting during sudden stops” issue.

Level Playability Testing: Adjusted platform spacing and collectible placement for reasonable jump rhythm and clear guidance.

Breakpoint Debugging: I extensively used Rider's breakpoint debugging features to locate root causes of logic issues.

Set breakpoints in state machine switching logic (`EntityManager<T>.Change`) to step through the call stack during transitions, verifying the trigger order of `onExit` and `onEnter` events, which helped identify that `BrakePlayerState` was not being triggered correctly.

Set breakpoints in ground detection (`HandleGround`) and gravity calculation (`Gravity`) to monitor runtime changes in `isGrounded` and `verticalVelocity`.

Discovered that `isGrounded` was not being updated correctly, preventing the character from landing. By tracing the execution order via breakpoints, I found

that HandleGround was executing after HandleController, causing ground detection to use position data from the previous frame. Fixed by moving HandleGround to execute before HandleController.

Used Rider's Step Over / Step Into functionality to trace through the velocity calculation logic in the Accelerate method line by line, verifying that velocity decomposition (forward velocity + turning velocity) behaved as expected and that Mathf.Clamp correctly limited maximum speed.

Combined Debug.Log output in Unity's Console with breakpoint debugging to form a complete debugging loop. For intermittent issues, used Debug.Break() to automatically pause the editor when specific conditions were met, allowing immediate inspection of the current frame's state.

These testing, debugging, and breakpoint-tracing processes were carried out throughout the entire development cycle, ensuring the stability of the final deliverable.

5. Evolution from Concept to Final Version

The project started from a simple technical prototype: a cube moving and jumping on a flat plane.

Phase 1: Implemented core movement and jumping. Established CRTP entity framework and basic state machine.

Phase 2: Refined feel and camera. Introduced acceleration, turning drag, Coyote Time, and double jump. Camera evolved to a complete third-person system with orbit, dead zones, and velocity-driven rotation.

Phase 3: Built the game loop. Added collectibles, checkpoints, goal, and victory conditions. Introduced audio system, UI, and settings panel.

Phase 4: Editor tools and designer-friendly design.

Developed [ClassName] dropdown selector. All parameters migrated to ScriptableObject.

The final version is a complete vertical slice: from gameplay to collection and victory, from sound feedback to settings—forming a closed player experience loop.

6. Personal Contribution

I completed all work on this project independently:

Game Concept & Design: Gameplay conception, mechanic design, experience

planning

Full Codebase: Entity framework, state machine, player controller, camera system, interaction systems, audio system, UI & settings, editor tools

Scene Building & Level Design: Platform placement, collectible layout, checkpoint and goal arrangement

Asset Integration & Configuration: Imported and configured third-party assets

Testing & Debugging: Continuous manual testing and iterative optimization

Core scripts authored:

EntityBase, Entity<T>, EntityState<T>, EntityStateManager<T>, Player, PlayerStats, PlayerInputManager, PlayerStateManager, PlayerCamera, PlayerAnimator, Game Manager, Coin, GameSavePoint, GameFinish, EffectManager, BgmManager, UI_Temp, SettingButtons, StopGame, ClassName, and ClassNameDrawer.

7. Use of Templates, Assets, Tutorials, and AI Support

Assets: They can be used freely for non-commercial purposes.

Player: <https://www.cgalpha.com/archives/68688.html>

Skybox: https://gitcode.com/open-source-toolkit/528bd/?utm_source=tools_gitcode&index=top&type=card&&uuid_tt_dd=10_18490143710-1775296053067-663627&isLogin=1&from_id=142948307&from_link=8f3698f0580f73da2a95c6bbe7608969

Platform: <https://kaylousberg.itch.io/block-bits/download/eyJleHBpcmVzIjoxNzg5MTA3MjYxLCJpZCI6MzM4OTc1NH0%3d.LMDxcbVBOSHK84av8NPuOzIf3Ww%3d>

Animal: <https://davidoreilly.itch.io/everything-library-animals>

Bottle: <https://indienova.com/resource/r/ddbxscyxsdb-2020low-poly-survival-pack>

Sound Effects: <https://assetstore.unity.com/packages/audio/sound-fx/rpg-essentials-sound-effects-free-227708>

BGM: <https://www.haimian.com/create>

AI: Used Haimian AI for background music generation(<https://www.haimian.com/create>).

Tutorials: Referenced Unity Learn tutorials on Input System and Cinemachine(<https://docs.unity.cn/cn/2022.3/Manual/UnityManual.html>).

Declaration: All third-party resources are used in accordance with their respective licenses. Non-commercially licensed resources are used solely for this course assignment and will not be used for any commercial purposes.